

Learning Report – Module 9: Application Debugging

Overview

This module helped me understand how to debug Angular applications using Google Chrome DevTools and Visual Studio Code. Debugging is an important part of software development because it helps identify and fix errors in an application. During this module, I learned how to inspect web pages, analyze TypeScript code, use breakpoints, and monitor the application's behavior to resolve issues effectively.

Learning Outcomes

After completing this module, I learned to:

- Debug Angular applications using Chrome DevTools.
- Inspect HTML elements using the DOM tree.
- Use the Sources panel to debug JavaScript and TypeScript code.
- Set breakpoints and inspect variables.
- Configure and use the Visual Studio Code debugger.
- Debug Angular components, services, and state management.
- Apply debugging techniques to improve application performance and reliability.

Debugging Angular with Chrome DevTools

Chrome DevTools is a built-in browser tool that helps developers inspect and debug web applications.

Inspecting Elements

Using the Elements tab, I learned how to inspect the HTML structure of an Angular application, view applied CSS styles, and identify layout or styling issues.

Using the Sources Panel

The Sources panel is used to debug JavaScript and TypeScript code. I learned how to set breakpoints, pause execution, step through code, and inspect variables to understand the application's execution flow.

Source Maps

Angular applications are written in TypeScript but executed as JavaScript. Source maps allow debugging the original TypeScript code instead of the compiled JavaScript, making debugging much easier.

Console and Network

The Console helps display logs and error messages, while the Network tab helps inspect API requests, responses, and loading times.

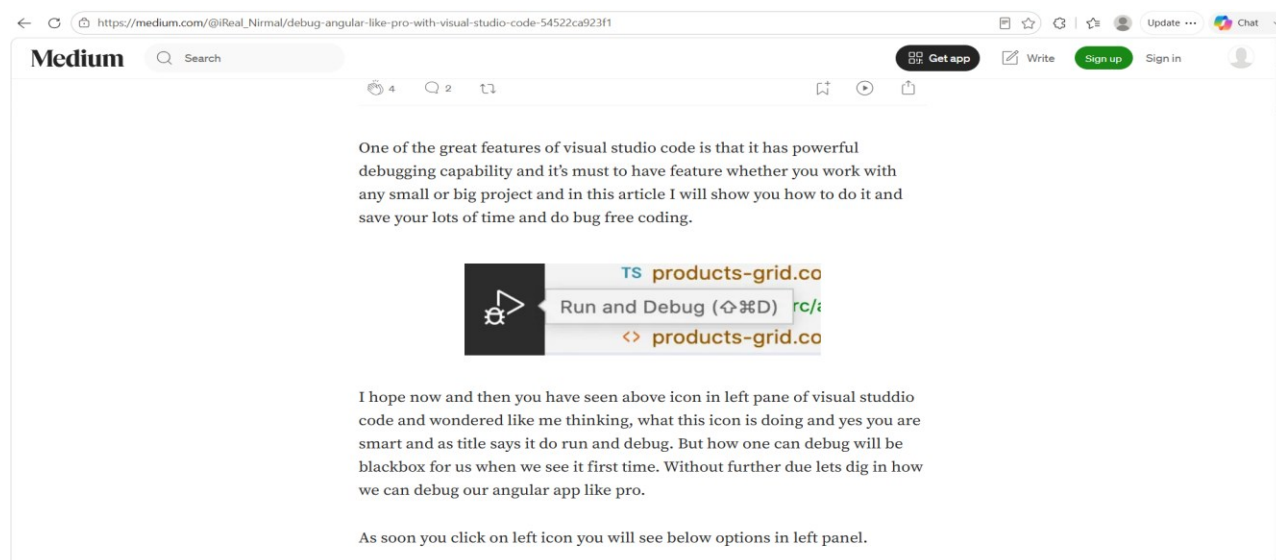
Debugging Angular in Visual Studio Code

Visual Studio Code provides an integrated debugging environment for Angular applications.

I learned how to:

- Configure the debugger using launch.json.
- Attach the debugger to a running Angular application.
- Set breakpoints in TypeScript files.
- Inspect variables and watch expressions.
- Use the Debug Console to evaluate expressions during execution.

Debugging directly in VS Code makes it easier to identify logical errors and monitor program execution.



Debugging Angular Components and Services

Angular applications contain components and services that manage the application's functionality.

During debugging, I learned to:

- Check component variables.
- Inspect service data.
- Debug API calls.
- Verify routing behavior.
- Monitor form validation.
- Debug asynchronous operations using Observables.

Debugging State Management

For applications using NgRx Store, I learned how to inspect actions, reducers, and application state. This helps in understanding how data flows through the application and locating state-related issues.

Best Debugging Practices

Some important debugging practices I learned include:

- Read error messages carefully.
- Use breakpoints instead of relying only on console logs.
- Check variable values while debugging.
- Verify API responses using the Network tab.
- Test the application after fixing each issue.
- Keep the code clean and organized for easier debugging.

Platforms and GenAI

This module also introduced supporting technologies used in modern software development:

- **Git** for version control.
- **CI/CD** for automated build and deployment.
- **Docker** for containerizing applications.
- **Generative AI** tools that assist in debugging, explaining errors, generating code, and improving developer productivity.

Conclusion

This module gave me a clear understanding of Angular debugging using Chrome DevTools and Visual Studio Code. I learned how to inspect the DOM, debug TypeScript code, analyze application behavior, and identify errors efficiently. I also understood the importance of debugging Angular components, services, API calls, and state management. Overall, this module improved my debugging skills and helped me understand how to develop more reliable and high-quality Angular applications.



[Courses](#) ▾[Tutorials](#) ▾[Practice](#) ▾[Jobs](#) ▾

Share Your Experiences

JavaScript Basics ▾

Array & String ▾

Function & Object ▾

OOP ▾

Asynchronous JavaScript ▾

Exception Handling ▾

DOM ▾

Advanced Topics ▾

Courses ▾

Summer SkillUp [Explore](#)

Error Handling and Debugging

Last Updated : Jan 10, 2025

Discuss 

Question 1

What is the purpose of the try...catch block in JavaScript?

A

 To handle asynchronous code

B

 To handle errors during code execution

C

 To stop the execution of a program

D

 To skip specific lines of code

Correct Answer!

Explanation

The try...catch block is used to catch runtime errors and execute custom error-handling logic.

Question 2

← ↻ 📄 https://www.toolsrail.com/quiz/web-browsers-developer-tools-quiz.php



[Home](#)[All Tools](#)[Math Quizzes](#)[Science Quizzes](#)[Engineering Quizzes](#)

[Home](#) / [Online Quizzes](#)

Web Browsers & Developer Tools Quiz

Test your browser debugging and DevTools skills with this fun and educational quiz built for developers.

 Quiz Options

Question Types

☒ Multiple Choice

☒ Shortcut Keys

☒ DevTools Panels

Difficulty

All Levels ▾

Browser Focus

All Browsers ▾

☒ Enable Hints

☒ Show Explanations

🕒 Question 1 of 10

🌟 Score: 0 / 10

🔍 Which keyboard shortcut activates the element selection mode in Chrome DevTools? ⓘ

Ctrl + E

Ctrl + Shift + E

Ctrl + Shift + C

Ctrl + Alt + S

← Previous

Next →

